CS5730 Operating Systems Midterm (2013/11/11)

Name:_____
ID#:_____

1 Short Answer: Your answers should be as concise as possible; aim for three or four sentences.

1.1 (6 pt) Explain why implementing synchronization primitives by disabling interrupts is not appropriate in a single-processor system if the synchronization primitives are to be used in user-level programs.

1.2 (6 pt) What is **busy-waiting** in the discussion of critical section problem? Explain how **busy-waiting** may degrade the system performance, or even deadlock?

1.3 (6 pt) In class, we discuss the bounded buffer problem which uses two semaphore, `empty` and `full`. Explain the usage of these two semaphores.

1.4 (5 pt) Illustrate how the two process scheduling criteria, **CPU utilization** and **throughput**, may conflict with each other?

1.5 (6 pt) What happens if one thread of a multi-threaded process call `fork()`? What happens if one thread of a multi-threaded process call `exec()`?

2 (10 pt) A variation of the round-robin scheduler is the **regressive round-robin scheduler**. This scheduler assigns each process a time quantum and a priority. The initial value of a time quantum is 50 milliseconds. However, every time a process has been allocated the CPU and uses its entire time quantum (does not block for I/O), 10 milliseconds is added to its time quantum, and its priority level is boosted. (The time quantum for a process can be increased to a maximum of 100 milliseconds.) When a process blocks before using its entire time quantum, its time quantum is reduced by 5 milliseconds, but its priority remains the same. What type of process (CPU-bound or I/O-bound) does the **regressive round-robin** scheduler favor? Explain.

3 (10 pt) Can a multi-threaded solution using multiple user-level threads achieve better performance on a multiprocessor system, which uses the many-to-one threading model, than on a single processor system? Explain.

4 Multilevel queue scheduling

4.1 (4 pt) List two possible scheduling among queues, which we discussed in class.

4.2 (5 pt) Why is it important for the scheduler to distinguish I/O-bound programs from CPU-bound programs?

4.3 (6 pt) Modify multilevel queue scheduling so it can distinguish I/O-bound

programs from CPU-bound programs.

5

5.1 (5 pt) Write to demonstrate how to use the atomic instruction `TestAndSet` for critical section problem.

5.2 (5 pt) Illustrate how your solution will fail if `TestAndSet` is not atomic.

5.3 (5 pt) Write to demonstrate how to use the atomic instruction `Swap` for critical section problem.

5.4 (5 pt) Illustrate how your solution will fail if `Swap` is not atomic.

6 (10 pt) **Race conditions** are possible in many computer systems. Consider a banking system with two methods: `deposit(amount)` and `withdraw(amount)`. These two methods are passed the amount that is to be deposited or withdrawn from a bank account. Assume that a husband and wife share a bank account and that concurrently the husband calls the `withdraw()` method and the wife calls `deposit()`.

7 (6 pt) Consider the following code segment:

```
pid_t pid;
pid = fork();
if (pid == 0) { /* child process */
    fork();
    thread_create( . . .);
}
fork();
```

a. How many unique processes are created?
b. How many unique threads are created?